BOXSPLOIT LABS / PRODUCT DOCUMENTATION

PANBOX

Panduan produk & operator

Control plane AppSec self-hosted. Instalasi, koneksi CI, kontrol akses, operasi, dan batas pilot yang jelas.

VERSION 0.1 · 17 OCTOBER 2026 · PILOT

© 2026 BOXSPLOIT LABS

Daftar isi

1. Komponen dan lokasi kerja

2. Pilih jalur setup

3. Jalankan dashboard di Debian/Linux/macOS

4. Jalankan dashboard di Windows

5. Tambahkan hanya agent ke CI

6. Hubungkan SCM publik ke PANBOX privat

7. Batasi akses sesuai kebutuhan

8. Kelola API key dan access key

9. Periksa dan kontrol scan di dashboard

10. Operasikan sesuai batas pilot

11. Telusuri langkah yang gagal

12. Pahami batas alpha

Panduan ini menjelaskan perilaku pilot yang sudah ada. Rencana pengembangan diberi label terpisah.

© 2026 BOXSPLOIT LABS

Mulai dari peranmu: operator server menyiapkan dashboard privat sekali; DevOps menambahkan satu langkah agent-only pada setiap job CI. Agent adalah satu binary Go. Runner tidak perlu Node, Python, Semgrep, Grype, atau instalasi server PANBOX.

1. Komponen dan lokasi kerja

```
Host Git/CI -> runner terlindungi -> agent PANBOX -> API PANBOX privat -> PostgreSQL
                                     | VACCIN: aturan source       |
                                     | ENETRON: paket -> OSV       +-> UI React
```

Hasil: CI menerima status pass, gate gagal, scan tidak lengkap, atau akses ditolak; operator melihat snapshot repo dan branch. File source tetap di runner. ENETRON mengirim nama dan versi paket ke API advisory kompatibel OSV. Agent mengunggah temuan ternormalisasi dengan API key PANBOX, tanpa potongan source mentah. Lihat diagram arsitektur dan jaringan.

2. Pilih jalur setup

- Operator server, sekali: siapkan Docker Engine/Desktop dengan Linux containers dan Compose v2. Ikuti bagian 3 untuk Debian/Linux/macOS atau bagian 4 untuk Windows. Debian adalah target produksi awal.
- DevOps, per repo: daftarkan repo, buat API key dengan scans:write, isi secret CI, lalu jalankan bagian 5. Runner perlu HTTPS keluar ke GitHub Releases dan API advisory, serta jalur ke API PANBOX privat.
- Tanpa IP/domain publik: pilih jalur koneksi pada bagian 6 sebelum menulis job CI.

PANBOX mengikat HTTP 8080 dan PostgreSQL 5434 ke loopback host. Ubah PANBOX_HOST_PORT atau PANBOX_DB_HOST_PORT di .env jika perlu. Jangan buka PostgreSQL ke internet.

3. Jalankan dashboard di Debian/Linux/macOS

Tujuan: hidupkan API dan UI privat sekali di host server:

```
git clone https://github.com/alterwriter/PANBOX.git
cd PANBOX
sh scripts/init-local.sh
docker compose --env-file .env -f deploy/docker-compose.yml up -d --build
curl -fsS http://127.0.0.1:8080/readyz
```

Hasil yang diharapkan: /readyz berhasil. Buka http://127.0.0.1:8080 di host server lalu login dengan PANBOX_BOOTSTRAP_EMAIL dan PANBOX_BOOTSTRAP_PASSWORD dari .env yang diabaikan Git. Initializer membuat secret dengan izin terbatas dan menolak menimpa .env. Kredensial bootstrap hanya berlaku saat database pertama kali dibuat; simpan volume pada restart rutin.

Demo opsional: PANBOX_SERVER=http://127.0.0.1:8080 bash scripts/seed-demo.sh memerlukan curl dan jq, lalu meminta password admin. Temuan demo adalah data fiktif.

4. Jalankan dashboard di Windows

Tujuan: jalankan container Linux yang sama melalui Docker Desktop. Di PowerShell:

```
git clone https://github.com/alterwriter/PANBOX.git
cd PANBOX
./scripts/init-local.ps1
docker compose --env-file .env -f deploy/docker-compose.yml up -d --build
Invoke-RestMethod http://127.0.0.1:8080/readyz
```

Hasil yang diharapkan: dashboard dan API sama seperti bagian 3. Jalankan ./scripts/seed-demo.ps1 hanya untuk data demo opsional. Saat membuat .env baru, pakai satu initializer saja.

© 2026 BOXSPLOIT LABS

5. Tambahkan hanya agent ke CI

Model onboarding: Daftarkan proyek PANBOX sebelum scan pertama. Project ID adalah penanda tujuan, bukan secret. Satu API key dengan scans:write dipakai untuk mengambil policy dan mengunggah hasil. Repo SCM privat tidak perlu dijadikan publik: kredensial checkout bawaan CI mengambil source, sementara PANBOX hanya menerima hasil ternormalisasi.

Sekali per repo, sebelum mengubah pipeline:

1. Masuk sebagai admin atau user dengan repos:write, buka Repositories -> Enroll repository. Pilih GitHub, GitLab, Bitbucket, Azure Repos, Gitea, atau Other Git.

2. Isi name (label tampilan, misalnya PANBOX) dan SCM repository ID or path (misalnya alterwriter/PANBOX; GitLab dapat memakai group/subgroup/project). Klik Test SCM connectivity and detect branch. Metadata publik GitHub, GitLab, Bitbucket, Azure Repos, atau Gitea yang diizinkan operator mengisi default branch otomatis. Owner boleh kosong; ini hanya label tampilan, bukan RBAC atau izin scan. Nama tampilan, path SCM, dan ID internal PANBOX adalah tiga nilai berbeda.

3. Baca hasil tes sebelum klik Enroll repository. Verified metadata berarti host PANBOX membaca default branch. Waiting for CI berarti metadata privat/tidak terjangkau; biarkan Auto-detect. Upload CI pertama dapat melaporkan default branch sebenarnya meski memindai feature branch. PANBOX tidak menebak branch scan pertama sebagai default. Untuk Git self-hosted, tambahkan host HTTPS persis ke PANBOX_SCM_ALLOWED_HOSTS. Salin PANBOX ID for CI ke PANBOX_REPOSITORY_ID; ID internal ini berbeda dari path SCM atau project ID GitLab.

4. Klik Create CI API key. Admin/key manager membutuhkan tokens:manage, scans:write, dan akses proyek. Gunakan scans:write, batasi ke proyek PANBOX ini jika perlu, lalu simpan secret yang tampil sekali sebagai secret CI terlindungi PANBOX_API_KEY. Tidak perlu repository token lain. Pemilik key tetap perlu membership proyek kecuali admin.

5. Buka Scan policy pada baris repo: aktifkan scan, pilih VACCIN/ENETRON, dan ambang gate. Atur PANBOX_SERVER ke URL API yang dijangkau runner. 127.0.0.1 hanya benar jika runner dan server satu host. Jangan taruh API key di file repo atau job pull request yang tidak tepercaya.

6. Dari runner dengan kredensial checkout SCM milik CI, pastikan checkout berhasil (git rev-parse --is-inside-work-tree), lalu cek koneksi dashboard, key, dan ID dengan perintah berikut. Tes metadata dashboard pada langkah 2 tidak membuktikan jalur runner. Setelah berhasil, jalankan job agent di bawah. Scan lengkap pertama muncul di Scans, lalu temuannya di Findings.

```
curl -fsS "$PANBOX_SERVER/readyz"
curl -fsS -H "Authorization: Bearer $PANBOX_API_KEY" \
  "$PANBOX_SERVER/v1/repos/$PANBOX_REPOSITORY_ID/scan-config"
```

Branch setelah enrollment: tiap job melaporkan branch checkout sendiri; feature branch muncul setelah CI menjalankan PANBOX di situ. GitLab CI_DEFAULT_BRANCH dan PANBOX_DEFAULT_BRANCH pada template GitHub melaporkan default sebenarnya walau job berjalan dari feature branch. CI lain dapat mengisi PANBOX_DEFAULT_BRANCH atau --default-branch. Tanpa metadata, default tetap pending. Job GitLab contoh hanya berjalan pada branch terlindungi; perluas rules untuk branch lain. Overview memakai default branch sebagai baseline. Recheck SCM memperbarui metadata otomatis.

Perbaiki enrollment yang salah: klik baris repo untuk detail. Edit mengubah nama, owner, dan default branch; gunakan Manual override bila perlu. Untuk GitHub Enterprise atau GitLab mandiri, isi origin HTTPS dan allowlist host persisnya di PANBOX_SCM_ALLOWED_HOSTS. Identitas SCM dapat diubah sampai scan pertama, lalu dikunci agar riwayat konsisten. Remove oleh Admin menghapus enrollment yang belum dipindai atau mengarsipkan yang sudah dipindai dan mencabut key terikat proyek. Restore mengaktifkannya lagi; gunakan API key dengan akses proyek. Owner kosong tidak memberikan izin.

Rule VACCIN native kustom: commit .panbox/rules.json di repositori yang dipindai. PANBOX membacanya di runner bersama rule bawaan; file yang tidak valid membuat modul VACCIN gagal, bukan diam-diam lulus. Contoh:

```
{"rules":[{"id":"custom.team.unsafe-call","severity":"medium","message":"Unsafe call needs
review","pattern":"unsafeCall\\(","extensions":[".js"]}]}
```

Pola JSON di atas mewakili regex unsafeCall\(. ID dimulai custom.; ekstensi yang didukung .py, .js, .jsx, .ts, .tsx, dan .go. Batas: 64 rule, 256 karakter per pola, file 64 KiB. Rule memeriksa satu baris setelah penyaringan komentar/string sederhana; belum melakukan taint analysis. Review file rule seperti perubahan kode lain dan uji pada repo yang representatif.

Setiap job Linux/macOS: ambil installer versi tetap dari release PANBOX v0.1.0-alpha.6. Installer memilih OS/CPU, mengunduh hanya binary agent, memeriksa SHA256SUMS, lalu memasang di workspace job tanpa sudo:

```
PANBOX_VERSION=v0.1.0-alpha.6
curl -fsSL
"https://github.com/alterwriter/PANBOX/releases/download/${PANBOX_VERSION}/install-agent.sh" -o
install-panbox-agent.sh
sh install-panbox-agent.sh "$PANBOX_VERSION" .panbox/bin
.panbox/bin/panbox go --server "$PANBOX_SERVER" --repo "$PANBOX_REPOSITORY_ID" --path .
```

Hasil yang diharapkan: exit 0 scan lengkap/pass; 1 gate gagal; 2 scan/transport/konfigurasi tidak lengkap; 3 autentikasi atau izin ditolak. Salin contoh GitHub Actions atau GitLab CI. Untuk runner Windows, unduh install-agent.ps1, lalu jalankan ./install-agent.ps1 -Version v0.1.0-alpha.6; hasilnya ./.panbox/bin/panbox.exe setelah verifikasi checksum. Binary agent langsung juga tersedia.

VACCIN memeriksa beberapa pola source Python, JavaScript/TypeScript, dan Go. ENETRON mencocokkan dependensi berversi tetap dari npm lockfile v2/v3, requirements.txt, dan go.mod dengan data advisory OSV. OSV adalah sumber data, bukan engine scanner. PANBOX_ADVISORY_URL atau --advisory-url memilih mirror kompatibel OSV. --offline meniadakan upload ke dashboard, tetapi ENETRON tetap menghubungi API advisory kecuali hanya --modules vaccin yang dipilih.

Urutan CI ke dashboard: (1) runner mengunduh agent versi tetap; (2) panbox go mengakses GET /v1/repos/{id}/scan-config dengan API key CI; (3) agent memindai modul aktif pada checkout; (4) mengirim satu bundle JSON ke POST /v1/scans dengan Authorization: Bearer $PANBOX_API_KEY; (5) API menyimpan snapshot dan mengembalikan keputusan gate; (6) agent keluar 0 (lulus), 1 (gate gagal), 2 (tidak lengkap/jaringan), atau 3 (autentikasi). Tidak ada setup upload terpisah. Cek policy dari runner: curl -fsS -H "Authorization: Bearer $PANBOX_API_KEY" "$PANBOX_SERVER/v1/repos/$PANBOX_REPOSITORY_ID/scan-config".

6. Hubungkan SCM publik ke PANBOX privat

- Runner lokal terlindungi (disarankan): polling SCM apa pun lewat HTTPS keluar, lalu mengirim ke PANBOX di LAN. Tidak perlu port PANBOX publik; gate sinkron. Contoh GitHub dan GitLab.
- SSH relay: PANBOX membuka reverse tunnel keluar ke relay yang terjangkau; shared runner membuka forward tunnel. PANBOX tidak menerima port internet masuk; gate sinkron. Pin host key dan batasi identity. Lihat contoh relay GitLab.
- Impor artifact: CI menulis panbox-results.json; host privat mengambil dan mengimpornya belakangan. Tanpa jalur masuk/relay; gate belakangan tidak dapat memblokir job CI itu. Contoh artifact GitLab dan panbox relay-pull mengotomasi GitLab; SCM lain memerlukan langkah unduh artifact sebelum impor generik panbox go --bundle.

Cek: dari runner sinkron, curl -fsS "$PANBOX_SERVER/readyz" harus berhasil. Repo publik tidak otomatis membuat artifact CI publik.

7. Batasi akses sesuai kebutuhan

Setup admin: Settings & Audit -> RBAC -> buat role dengan izin eksplisit -> assign user -> beri membership repo. admin adalah role tertinggi dan tidak dapat diedit. Role bawaan: security_reviewer, developer, executive_viewer, dan scanner_ci; role custom tidak dapat diberi rbac:manage.

Cek: akses data memerlukan izin dan membership repo. Izin pilot: repos:read, repos:write, scans:read, scans:write, findings:read, findings:recast, reports:export, rbac:manage, tokens:manage, dan audit:read.

© 2026 BOXSPLOIT LABS

8. Kelola API key dan access key

Lokasi: Settings & Audit -> API & access keys. Pilih label, tipe, expiry, scope, dan repo bila perlu. API key memiliki scope integrasi terpilih; access key hanya baca; API key CI memakai scans:write dan scope proyek opsional. Salin secret yang tampil sekali ke secret manager. Key disimpan sebagai hash, punya masa berlaku, dan dapat dicabut.

Tes API baca:

```
curl -H "Authorization: Bearer $PANBOX_ACCESS_KEY" "$PANBOX_SERVER/v1/repos"
```

Browser memakai password, session cookie HttpOnly, dan CSRF token. Kontrak OpenAPI memuat endpoint pilot. Gunakan Rotate untuk membuat secret pengganti dan mencabut key lama secara atomik; perbarui secret CI sebelum job berikutnya.

9. Periksa dan kontrol scan di dashboard

Periksa dan triase: klik baris repo untuk detail serta aksi langsung View findings / View scans. Klik scan untuk daftar temuannya, lalu klik finding untuk path, baris, URL source bila tautan commit aman dibentuk, advisory, petunjuk remediasi, dan 100 scan lengkap terakhir pada branch itu. Temuan yang hilang pada scan lengkap berikutnya untuk modulnya berstatus resolved; riwayat lama tetap terlihat. Recast severity mengubah effective severity dengan alasan yang diaudit. Triage dapat menandai false positive, accepted risk, closed, atau reopen, juga dengan alasan; status ini merevisi gate terkait dan mengeluarkan temuan dari hitungan posture aktif sambil mempertahankannya dalam riwayat/laporan. Penutupan manual tetap berlaku sampai dibuka lagi. Tombol prompt remediasi hanya menyalin template lokal; tidak memanggil AI atau mengubah source. Bridge MCP hanya-baca dapat memasok finding ke klien, tetapi klien yang mengatur penggunaan AI. Persentase false positive pilot ini belum diukur.

Recast: buka Findings, kosongkan filter bila daftar belum terlihat, lalu klik Recast severity di samping rule temuan (kolom pertama). Pilih effective severity atau pulihkan severity scanner -> isi alasan (10-500 karakter) -> Save. Tombol hanya terlihat jika akun memiliki findings:recast dan akses repo; admin atau security_reviewer dengan membership repo dapat memakainya. Jika tidak ada baris, selesaikan scan pertama atau periksa filter. Jika ada baris tetapi tidak ada tombol, minta admin memeriksa role dan membership di Settings & Audit -> RBAC. Raw severity tidak berubah. Riwayat tersedia di GET /v1/findings/{id}/recasts; gate lama memperoleh revisi dengan ambang asli masing-masing scan. Recast bukan persetujuan atau pengabaian temuan.

Ekspor: Reports -> pilih repo, branch tepat, modul scan (VACCIN/ENETRON), dan severity opsional -> Export PDF, CSV, atau JSON. Data berasal dari scan lengkap terbaru tiap branch. PDF adalah ringkasan teknis; CSV/JSON menyimpan kolom lengkap. Perlu reports:export.

MCP: Settings & Audit -> API & access keys -> buat Access key hanya-baca dengan repos:read, scans:read, dan findings:read. Buka Settings & Audit -> MCP setup dan ikuti konfigurasi host stdio generik. Host MCP menjalankan panbox mcp dengan PANBOX_SERVER dan PANBOX_TOKEN di secret environment. Empat tool hanya-baca menampilkan repo, temuan, daftar scan, dan detail scan. Tidak ada endpoint MCP publik.

Setelah CI: Overview menampilkan cakupan/severity; Repositories menampilkan proyek; Scans menampilkan status run/gate; Findings dapat difilter menurut repo/branch/modul/severity; Reports mengekspor PDF/CSV/JSON teknis terfilter; Settings & Audit menampilkan RBAC, key, dan audit. Tema light dan dark tersedia.

Arti hasil: scan parsial tetap unverified dan mempertahankan posture sukses terakhir. ARK menandai Critical/High dan BUG-STER menandai Medium/Low, sementara severity standar tetap disimpan dan diekspor. Recast dan perubahan scan policy tercatat di audit.

10. Operasikan sesuai batas pilot

Biarkan PostgreSQL di loopback. Untuk browser jarak jauh, gunakan reverse proxy HTTPS dengan kontrol akses atau jalur privat; set PANBOX_SECURE_COOKIE=true pada HTTPS. Backup PostgreSQL dan uji restore.

© 2026 BOXSPLOIT LABS

Host Debian 2 GB adalah target untuk divalidasi, bukan jaminan. Fixture kecil Docker Desktop Windows mengukur sekitar 5,1 MiB RAM server dan 37,3 MiB PostgreSQL; 100 panggilan API berurutan menghasilkan median 7,1 ms dan p95 9,3 ms. Agent berada di runner dan butuh sizing terpisah. Ukur docker stats --no-stream dengan repo dan bundle yang representatif.

11. Telusuri langkah yang gagal

- Dashboard belum siap: jalankan docker compose --env-file .env -f deploy/docker-compose.yml ps dan logs db server; periksa kesehatan PostgreSQL.
- Unduhan agent gagal: periksa tag release, nama asset OS/CPU, akses GitHub keluar, curl, dan alat SHA-256. Jangan lewati checksum.
- Runner gagal menjangkau PANBOX: 127.0.0.1 di container menunjuk ke container itu; gunakan alamat host/jaringan privat atau SSH relay.
- Exit 2: cek manifest, endpoint advisory, path kerja, endpoint PANBOX, dan log; tidak lengkap bukan pass.
- Exit 3: cek expiry/revocation API key, scans:write, ID repo internal, role, dan membership.
- Impor artifact terlambat: cek timer, GITLAB_READ_TOKEN, ref yang diizinkan, Jobs API/artifact, dan state relay.

12. Pahami batas alpha

Yang tersedia: server/UI, pemeriksaan source native VACCIN, pencocokan dependensi tetap native ENETRON dengan data OSV, snapshot repo/branch, gate, RBAC, key berscope, audit, ekspor PDF/CSV/JSON terfilter, serta contoh jalur koneksi di atas. Validasi runner nyata, OSV/mirror, beban Debian, backup/restore, TLS, dan rotasi key di lingkungan target.

Cakupan masih sempit. RIDER secrets, TESLA IaC, alur persetujuan, template PDF eksekutif, AI Security, OIDC, dan rate limiting API masih roadmap. PDF ini panduan produk, bukan laporan kerentanan PANBOX.

© 2026 BOXSPLOIT LABS